```
import java.util.ArrayList;

public class Gambler extends Person implements CardPlayerI, DicePlayerI {

    ArrayList<Card> cardsInHand = new ArrayList<>();

    public Gambler(String id, String fName, String sName, LocalDate
birthday) {
        super(id, fName, sName, birthday);
    }

    @Override
    public Card pickTop(Deck deck) {
        Card card = deck.pickCard();
        cardsInHand.add(card);
        return card;
    }

    @Override
    public int roll(Dice dice) {
        return dice.roll();
    }

    @Override
    public ArrayList<Card> openCards() {
        ArrayList<Card> rVal = new ArrayList<>(cardsInHand);
        cardsInHand.clear();
        return rVal;
    }
}
```

Κώδικας 14.7 *Η κλάση Gambler*

Η κλάση Gambler στον κώδικα 14.7 κληρονομεί από την Person (κώδικας 12.19) άρα διαθέτει τους τύπους, τις υλοποιήσεις και τις μεταβλητές της Person. Κάθε αντικείμενο Gambler επομένως είναι ένα Person. Επιπλέον υλοποιεί τις διεπαφές CardPlayerI και DicePlayerI. Αυτό σημαίνει πως για να είναι σαφής και όχι αφηρημένη, η Gambler πρέπει να παρέχει υλοποιήσεις για τις αφηρημένες μεθόδους των διεπαφών. Επομένως, κάθε αντικείμενο Gambler μπορεί να επιτελέσει τις λειτουργίες που ορίζονται στις CardPlayerI και DicePlayerI. Συνεπώς, κάθε αντικείμενο Gambler θεωρείται πως είναι ένας παίκτης ζαριών και ένας χαρτοπαίκτης. Ο κώδικας 14.8 επιδεικνύει τη χρήση της Gambler.

```
import java.time.LocalDate;
import java.time.Month;

public class GamblerDemo {

    public static void main(String[] args) {
        Deck deck = new Deck();
        double[] fairDist = new double[]{1d / 6, 1d / 6, 1d / 6, 1d / 6, 1d
/ 6, 1d / 6,};
        Dice dice = new Dice(fairDist);

        Gambler jim = new Gambler("01", "Jim", "Black", LocalDate.of(1976,
Month.JULY, 5));
        Gambler jack = new Gambler("02", "Jack", "Green",
LocalDate.of(1981, Month.APRIL, 12));
        deck.shuffle();
        for (int i = 0; i < 4; i++) {
            jim.pickTop(deck);
            jack.pickTop(deck);
        }
```

```
            System.out.println("Οι κάρτες του " + jim.getsName() + " " +
    jim.openCards());
            System.out.println("Οι κάρτες του " + jack.getsName() + " " +
    jack.openCards());
            System.out.println("Ζαριά του " + jim.getsName() + " " +
    jim.roll(dice));
            System.out.println("Ζαριά του " + jack.getsName() + " " +
    jack.roll(dice));
            CardPlayerI jimCP = jim;
            DicePlayerI jackDP = jack;
            Person jimP = jim;
        }
    }
```

Κώδικας 14.8 *Κάθε αντικείμενο της κλάσης Gambler είναι Person και DicePlayerI και CardPlayerI*

Εδώ ορίζουμε δύο αντικείμενα τύπου Gambler. Στη συνέχεια ανακατεύουμε μια τράπουλα και τους δίνουμε να τραβήξουν από τέσσερις κάρτες ο κάθε ένας. Μετά ανοίγουν τις κάρτες τους τις οποίες εμφανίζουμε συνοδευόμενες από το επώνυμο του αντίστοιχου παίκτη. Επομένως, μέχρι εδώ είναι σαφές πως κάθε αντικείμενο Gambler είναι ένα πρόσωπο και ένας χαρτοπαίκτης ταυτοχρόνως. Στο επόμενο βήμα τους βάζουμε να ρίξουν από μία ζαριά. Ας σημειωθεί πως η κλήση Dice dice = new Dice(fairDist) θα μπορούσε να αντικατασταθεί με την κλήση Dice dice=new Dice() με ίδιο αποτέλεσμα.

Οι τελευταίες τρεις γραμμές της main έχουν ως σκοπό να επιδείξουν πως αυτά τα αντικείμενα μπορούμε να τα δούμε μέσα στον κώδικά μας αποκλειστικά είτε σαν Person είτε σαν DicePlayer είτε σαν CardPlayer.

14.4 Άλλα χαρακτηριστικά

14.4.1 Κληρονομικότητα διεπαφών

Οι διεπαφές δεν μπορούν να κληρονομήσουν από κλάσεις. Ο κώδικας

```
interface Student extends Person {}
```

παράγει λάθος μεταγλώττισης.

Οι διεπαφές μπορούν να κληρονομήσουν από απεριόριστο αριθμό διεπαφών. Ο κώδικας

```
public interface Gambler extends CardPlayerI, DicePlayerI {}
```

είναι ορθός.

14.4.2 Η κλάση νικάει

Στην περίπτωση που μία κλάση κληρονομεί τον ίδιο τύπο από κλάση και διεπαφή, τότε κληρονομείται η υλοποίηση της κλάσης και ο κοινός τύπος δεν είναι υποχρεωτικό να επανοριστεί στην παράγωγη κλάση.

```
import java.time.LocalDate;
import java.time.Month;

interface Named {
    public String getfName();
    public String getsName();
}

public class Inherited extends Person implements Named {

    public Inherited(String id, String fName, String sName, LocalDate
birthday) {
```

```
            super(id, fName, sName, birthday);
        }

        public static void main(String[] args) {
            Inherited i=new Inherited("1", "George", "Best",
    LocalDate.of(1950,Month.MARCH, 1));
            System.out.println(i.getfName());
        }
    }
```

Κώδικας 14.9 *Η κλάση Inherited κληρονομεί ίδιους τύπους από τη διεπαφή Named και την κλάση Person*

Η κλάση Inherited είναι παράγωγη της Person. Επομένως κληρονομεί τις μεθόδους της Person ανάμεσα στις οποίες συμπεριλαμβάνονται οι getfName() και getsName(). Παράλληλα, η κλάση υλοποιεί τη διεπαφή Named που ορίζει τις ίδιες μεθόδους. Σε αυτήν την περίπτωση ισχύει ο κανόνας που λέει πως η κλάση νικάει (calss wins), δηλαδή η Inherited κληρονομεί τις υλοποιήσεις της Person. Επομένως, έχει τη δυνατότητα να τις επανορίσει χωρίς όμως να είναι υποχρεωτικό.

14.4.3 Στατικά μέλη

Τα στατικά μέλη δεν κληρονομούνται. Τα στατικά μέλη μπορούν να οριστούν στις διεπαφές με τον ίδιο τρόπο που ορίζονται και στις κλάσεις. Η προσπέλαση σε αυτά γίνεται με προσδιοριστή ονόματος το όνομα της διεπαφής ακολουθούμενο από τον τελεστή τελεία και το όνομα του μέλους.

```
import java.util.Arrays;

public class AllaXaraktiristika {

    public interface Gambler extends CardPlayerI, DicePlayerI {
    }

    public interface Distribution {

        public double[] getDistribution();

        public static Double[] fairDistribution() {
            return new Double[]{1d / 6, 1d / 6, 1d / 6, 1d / 6, 1d / 6, 1d
/ 6};
        }
    }

    public static void main(String[] args) {
        Double[] d = Distribution.fairDistribution();
        System.out.println(Arrays.asList(d));
    }
}
```

Κώδικας 14.10 *Στατικά μέλη στις διεπαφές*

Όπως φαίνεται στον κώδικα 14.10, η διεπαφή Distribution ορίζει το αφηρημένο μέλος getDistribution και το στατικό μέλος fairDistribution. Εφόσον η fairDistribution είναι μία για όλες τις υλοποιήσεις της διεπαφής, λογικό είναι να ορισθεί ως στατική. Η fairDistribution μπορεί να κληθεί ως Distribution. fairDistribution().

14.4.4 Προκαθορισμένες μέθοδοι

Οι προκαθορισμένες μέθοδοι (default methods) στις διεπαφές είναι αποτέλεσμα της πολιτικής της Java να διατηρεί συμβατότητα με όλες τις προηγούμενες εκδόσεις της. Θα το εξηγήσουμε αυτό με το παράδειγμα της διεπαφής Iterator. Αρχικά, η διεπαφή όριζε δύο αφηρημένες μεθόδους, την hasNext() και την next(). Κάποια στιγμή έγινε αντιληπτό πως η πρόσθεση μιας μεθόδου remove() που διαγράφει από την υποκείμενη συλλογή το τρέχον στοιχείο του Iterator θα ήταν χρήσιμη σε πολλές περιπτώσεις. Έως ότου να προστεθεί όμως η remove

είχε αναπτυχθεί παγκοσμίως σημαντικός όγκος κώδικα που χρησιμοποιούσε τον Iterator υλοποιώντας μόνο την hasNext() και την next(). Εάν λοιπόν προστίθετο στην Iterator η remove ως αφηρημένη μέθοδος, ο μεταγλωττιστής θα απαιτούσε την υλοποίησή της με αποτέλεσμα ο προϋπάρχων κώδικας να μην μεταγλωττίζεται. Ωστόσο ο προϋπάρχων κώδικας δεν αξιοποιούσε την remove, καθώς όταν αναπτύχθηκε δεν υπήρχε. Προκειμένου να αντιμετωπιστεί αυτό το πρόβλημα εισήχθησαν στην έκδοση 8 της Java οι προκαθορισμένες μέθοδοι. Οι προκαθορισμένες μέθοδοι παρέχουν υλοποίηση έτσι ώστε οι κώδικες-πελάτες να περνούν μεταγλώττιση χωρίς να απαιτείται να τις υλοποιήσουν, ενώ παράλληλα νέοι κώδικες που τις χρειάζονται μπορούν να τις επανορίσουν.

Σημειώστε πως η λειτουργικότητα των προκαθορισμένων μεθόδων δεν επαρκεί για να χρησιμοποιηθούν παρά μόνο αν επανοριστούν. Πιο συγκεκριμένα, οι προκαθορισμένες μέθοδοι, εφόσον δεν επανοριστούν, απλά παράγουν μια εξαίρεση που πληροφορεί πως η μέθοδος δεν έχει οριστεί.

Τέλος να σημειωθεί πως ο κανόνας η κλάση νικάει ισχύει και για τις προκαθορισμένες μεθόδους.

14.4.5 Ανώνυμες κλάσεις

Μερικές φορές χρειαζόμαστε μια κλάση μόνο μία φορά μέσα στο πρόγραμμά μας. Σε αυτές τις περιπτώσεις, οι διεπαφές μας δίνουν τη δυνατότητα αξιοποίησης ανώνυμων (anonymous) κλάσεων. Αν υποθέσουμε πως χρειαζόμαστε ένα μόνο αντικείμενο τύπου DicePlayerI, τότε μπορούμε να το δημιουργήσουμε όπως δείχνει ο κώδικας 14.11, δηλαδή με τον τελεστή new ακολουθούμενο από τη διεπαφή DicePlayerI και ένα μπλοκ στο οποίο επανορίζουμε τις μεθόδους της διεπαφής και προσθέτουμε οποιαδήποτε άλλα μέλη χρειαζόμαστε. Με αυτόν τον τρόπο έχουμε ένα αντικείμενο με τα χαρακτηριστικά που χρειαζόμαστε, εκεί ακριβώς που το χρειαζόμαστε. Ωστόσο, σε γενικές γραμμές δεν συνίσταται η χρήση μεγάλων και πολύπλοκων ανώνυμων κλάσεων.

```
public class AnonymousDemo {

    public static void main(String[] args) {
        Dice dice = new Dice();
        DicePlayerI jim = new DicePlayerI() {
            @Override
            public int roll(Dice dice) {
                return dice.roll();
            }
        };
        for (int i = 0; i < 50; i++) {
            System.out.println(jim.roll(dice));
        }
    }
}
```

Κώδικας 14.11 *Επίδειξη της ανώνυμης κλάσης*

14.5 Λυμένες Ασκήσεις

14.5.1. Καζίνο

Σε ένα καζίνο υπάρχει η δυνατότητα να παίξει κανείς διαφορετικά παιχνίδια με τράπουλα. Κάποια παιχνίδια απαιτούν και τις 52 κάρτες της τράπουλας και κάποια παίζονται με τράπουλα που περιέχει τις κάρτες από 7 έως τον άσο, δηλαδή με 32 κάρτες. Έτσι το καζίνο χρειάζεται 2 τύπους τράπουλας.
Δημιουργήστε κατάλληλη διεπαφή τράπουλας και δύο κλάσεις που υλοποιούν τη διεπαφή. Η μία κλάση θα αναπαριστά τράπουλα 52 καρτών και η άλλη τράπουλα 32 καρτών. Σε κάθε μία από αυτές τις κλάσεις, υλοποιήστε τον αποθηκευτικό χώρο ως πίνακα από κάρτες. Αναπτύξτε εφαρμογή που αποθηκεύει μια τράπουλα 32 καρτών και μία 52 καρτών σε κατάλληλο πίνακα. Στη συνέχεια για κάθε τράπουλα στον πίνακα, ανακατέψτε την, σηκώστε μια κάρτα, εμφανίστε την κάρτα και στη συνέχεια εμφανίστε την υπόλοιπη τράπουλα. Ελέγξτε την ορθότητα της εξόδου.

Λύση

Καταρχάς, παρουσιάζεται η διεπαφή:

```
public interface Deck {
    public void shuffle();
    public boolean empty();
    public Card pickTop();
}
```

Κώδικας 14.12 *Η διεπαφή Deck*

Οι μέθοδοι shuffle και pickTop απαιτούνται άμεσα από την εκφώνηση. Η μέθοδος empty που ελέγχει αν υπάρχει ή όχι στην τράπουλα κάρτα είναι γενικότερα χρήσιμη. Στη συνέχεια δίνεται η κλάση Deck52 που αναπαριστά τράπουλες των 52 καρτών.

```
import java.util.Random;

public class Deck52 implements Deck {

    final Card[] deck;
    private int cardsInDeck;
    private static final int DECKSIZE = 52;

    public Deck52() {
        deck = new Card[DECKSIZE];
        cardsInDeck = 0;
        for (CardRank r : CardRank.values()) {
            for (Suit s : Suit.values()) {
                deck[cardsInDeck++] = new Card(r, s);
            }
        }
    }

    @Override
    public void shuffle() {
        Random r = new Random();

        for (int i = 0; i < deck.length; i++) {
            int rIdx = r.nextInt(deck.length);
            Card tmp = deck[i];
            deck[i] = deck[rIdx];
            deck[rIdx] = tmp;
        }
    }

    @Override
    public boolean empty() {
        return cardsInDeck == 0;
    }

    @Override
    public Card pickTop() {
        cardsInDeck--;
        return deck[DECKSIZE - cardsInDeck - 1];
    }

    @Override
    public String toString() {
        if (cardsInDeck == 0) {
            return "Empty Deck";
        }
        StringBuilder rVal = new StringBuilder();
```

```
            for (int i = DECKSIZE - cardsInDeck; i < DECKSIZE; i++) {
                if (i > DECKSIZE - cardsInDeck && i % 4 == 0) {
                    rVal.append("\n");
                }
                rVal.append(deck[i] + " ");
            }
            return rVal.toString();
        }
    }
```

Κώδικας 14.13 *Η κλάση Deck52*

Παρόμοια είναι και η κλάση Deck32 που αναπαριστά τράπουλες των 32 καρτών.

```
    import java.util.Random;

    public class Deck32 implements Deck {

        private final Card[] deck;
        private int cardsInDeck;
        private static final int DECKSIZE = 32;

        public Deck32() {
            deck = new Card[DECKSIZE];
            cardsInDeck = 0;
            for (int i = 5; i < 13; i++) {
                CardRank r = CardRank.values()[i];
                for (Suit s : Suit.values()) {
                    deck[cardsInDeck++] = new Card(r, s);
                }
            }
        }

        @Override
        public void shuffle() {
            Random r = new Random();

            for (int i = 0; i < deck.length; i++) {
                int rIdx = r.nextInt(deck.length);
                Card tmp = deck[i];
                deck[i] = deck[rIdx];
                deck[rIdx] = tmp;
            }
        }

        @Override
        public boolean empty() {
            return cardsInDeck == 0;
        }

        @Override
        public Card pickTop() {
            cardsInDeck--;
            return deck[DECKSIZE - cardsInDeck - 1];
        }

        @Override
        public String toString() {
            if (cardsInDeck == 0) {
                return "Empty Deck";
            }
            StringBuilder rVal = new StringBuilder();
```

```
            for (int i = DECKSIZE - cardsInDeck; i < DECKSIZE; i++) {
                if (i > DECKSIZE - cardsInDeck && i % 4 == 0) {
                    rVal.append("\n");
                }
                rVal.append(deck[i] + " ");
            }
            return rVal.toString();
        }
    }
```

Κώδικας 14.14 *Η κλάση Deck32*

Στον κώδικα 14.15 δίνεται και η εφαρμογή που ζητείται από την εκφώνηση.

```
    public class App {
        public static void main(String[] args) {
            Deck[] decks=new Deck[2];
            decks[0]=new Deck32();
            decks[1]=new Deck52();
            for (Deck deck:decks) {
                deck.shuffle();
                System.out.println(deck.pickTop());
                System.out.println("------------------------------");
                System.out.println(deck);
                System.out.println("------------------------------");
            }
        }
    }
```

Κώδικας 14.15 *Η ζητούμενη εφαρμογή*

14.5.2 Τζόγος στου John

Στη συγκέντρωση που διοργάνωσε ο John Beker μαζεύτηκαν 14 πρόσωπα. Σε κάποιους από τους καλεσμένους αρέσει να παίζουν ζάρια. Ο John έχει 3 ζάρια. Κάποια ζάρια είναι κανονικά και φέρνουν κάθε αποτέλεσμα από 1 έως 6 με πιθανότητα 1/6, κάποια άλλα όμως δεν φέρνουν ποτέ 5 και φέρνουν 6 με πιθανότητα 1/3. Τέλος, υπάρχουν και ζάρια που δεν φέρνουν 4 ποτέ αλλά φέρνουν 5 με πιθανότητα 1/3.

Επίσης, κάποιοι από τους καλεσμένους παίζουν χαρτιά.

Στους καλεσμένους του John συμπεριλαμβάνονται κάποιοι που παίζουν αποκλειστικά ζάρια, κάποιοι που παίζουν αποκλειστικά χαρτιά αλλά και κάποιοι που παίζουν και χαρτιά και ζάρια.

Σε μια παρτίδα ζαριών συμμετέχουν 2 παίκτες. Ρίχνουν εναλλάξ από 10 ζαριές. Κάθε φορά που οι 2 παίκτες ρίχνουν το ζάρι είτε ο ένας φέρνει μεγαλύτερο αποτέλεσμα από τον άλλον είτε φέρνουν το ίδιο αποτέλεσμα. Στην πρώτη περίπτωση ενημερώνεται αθροιστικά το σκορ του παίκτη που έφερε το μεγαλύτερο αποτέλεσμα. Στη δεύτερη περίπτωση δεν ενημερώνεται κανένα σκορ.

Σε μια παρτίδα χαρτιών συμμετέχουν όλοι οι διαθέσιμοι (χαρτο)παίκτες. Στην αρχή της παρτίδας ανακατεύουν την τράπουλα και στη συνέχεια τραβάνε από μια κάρτα εναλλάξ έως ότου τελειώσουν οι κάρτες. Στο τέλος της παρτίδας, ο κάθε παίκτης παρουσιάζει τα χαρτιά που έχει μαζέψει.

Ζητείται να αναπτυχθεί εφαρμογή που:

1. Βάζει κάθε πρόσωπο στη συγκέντρωση του John να συναντηθεί με κάθε άλλο πρόσωπο. Αν κατά τη συνάντηση είναι και οι 2 παίκτες ζαριού, τότε παίζουν μια παρτίδα ζάρια. Αν όμως κατά τη συνάντηση κάποιος είναι χαρτοπαίκτης, τότε εγγράφεται στην ομάδα των χαρτοπαικτών.
2. Όλοι οι εγγεγραμμένοι χαρτοπαίκτες συμμετέχουν σε μια παρτίδα χαρτιών.
3. Εμφανίζει το ονοματεπώνυμο και σκορ για κάθε παίκτη ζαριού κατά φθίνουσα σειρά των σκορ.
4. Νικητής στα χαρτιά ανακηρύσσεται αυτός που έχει μαζέψει τους περισσότερους άσους. Αν 2 ή περισσότεροι παίκτες έχουν τον ίδιο μέγιστο αριθμό άσων, τότε ανακηρύσσονται νικητές.

Λύση

Καταρχάς, θα πρέπει τώρα να σχεδιάσουμε μια κλάση για τα πρόσωπα που είναι παίκτες ζαριού, μια για όσους είναι χαρτοπαίκτες και μία για όσους παίζουν και χαρτιά και ζάρια. Και οι τρεις κλάσεις είναι παράγωγες της κλάσης Person. Επιπλέον, η κλάση των χαρτοπαικτών υλοποιεί τη διεπαφή CardPlayerI, η κλάση των παικτών ζαριού υλοποιεί τη διεπαφή DicePlayerI και η κλάση των προσώπων που είναι και χαρτοπαίκτες και παίκτες ζαριού υλοποιεί και την DicePlayerI και την CardPlayerI. Την τελευταία κλάση, την έχουμε υλοποιήσει ήδη (κώδικας 14.7).

```
import java.time.LocalDate;
import java.util.ArrayList;

public class CardPlayer extends Person implements CardPlayerI {

    ArrayList<Card> cardsInHand = new ArrayList<>();

    public CardPlayer(String id, String fName, String sName, LocalDate
birthday) {
        super(id, fName, sName, birthday);
    }

    @Override
    public Card pickTop(Deck deck) {
        Card card = deck.pickCard();
        cardsInHand.add(card);
        return card;
    }

    @Override
    public ArrayList<Card> openCards() {
        ArrayList<Card> rVal = new ArrayList<>(cardsInHand);
        cardsInHand.clear();
        return rVal;
    }
}
```

Κώδικας 14.16 *Η κλάση CardPlayer*

Η κλάση CardPlayer διαθέτει ArrayList<Card> στο οποίο αποθηκεύεται κάθε κάρτα που λαμβάνει ο παίκτης από τράπουλα. Τα αντικείμενα δημιουργούνται με κλήση στον δημιουργό της γονικής κλάσης Person. Η pickTop λαμβάνει μια κάρτα από την τράπουλα που της δίνεται παραμετρικά, η openCards αντιγράφει τις κάρτες που έχει ο παίκτης στα χέρια του σε ένα άλλο ArrayList, στη συνέχεια, διαγράφει τις κάρτες στα χέρια του παίκτη και επιστρέφει το αντίγραφο.

```
import java.time.LocalDate;

public class DicePlayer extends Person implements DicePlayerI {

    public DicePlayer(String id, String fName, String sName, LocalDate
birthday) {
        super(id, fName, sName, birthday);
    }

    @Override
    public int roll(Dice dice) {
        return dice.roll();
    }
}
```

Κώδικας 14.17 *Η κλάση DicePlayer*

Ο κώδικας 14.73 παρουσιάζει την κλάση παικτών ζαριού. Η κλάση Dice έχει δοθεί ήδη στον κώδικα 14.6. Η κλάση Person είναι η έκδοση που δίνεται στον κώδικα 12.19. Με τις κλάσεις αυτές στη διάθεσή μας μπορούμε να προχωρήσουμε στην ανάπτυξη της εφαρμογής.

```
import java.time.LocalDate;
import java.time.Month;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.HashSet;
import java.util.LinkedHashMap;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Map.Entry;
import java.util.Random;
import java.util.Set;

public class BeckersMeeting {

    private static final Map<DicePlayerI, Integer> diceScores = new
LinkedHashMap<>();
    private static final Dice fair = new Dice();
    private static final Dice two6 = new Dice(new double[]{1. / 6, 1. / 6,
1. / 6, 1. / 6, 0, 1. / 3,});
    private static final Dice two5 = new Dice(new double[]{1. / 6, 1. / 6,
1. / 6, 0, 1. / 3, 1. / 6,});
    private static final Dice[] dices = new Dice[]{fair, two6, two5};
    private final static Random gen = new Random();

    static Dice pickDice() {
        return dices[gen.nextInt(3)];
    }

    private static void diceGame(DicePlayerI p1, DicePlayerI p2) {
        int score1 = 0;
        int score2 = 0;
        for (int i = 0; i < 10; i++) {
            int s1 = p1.roll(pickDice());
            int s2 = p2.roll(pickDice());
            if (s1 > s2) {
                score1 += s1;
            } else if (s2 > s1) {
                score2 += s2;
            }
        }

        Integer score = diceScores.get(p1);
        if (score == null) {
            score = 0;
        }
        diceScores.put(p1, score1 + score);

        score = diceScores.get(p2);
        if (score == null) {
            score = 0;
        }
        diceScores.put(p1, score2 + score);
    }

    private static void cardGame(Set<CardPlayerI> cardPlayers) {
```

```
        Deck deck = new Deck();
        deck.shuffle();
        while (!deck.empty()) {
            for (CardPlayerI cp : cardPlayers) {
                cp.pickTop(deck);
                if (deck.empty()) {
                    break;
                }
            }
        }
    }

    private static void meet(Person p1, Person p2, Set<CardPlayerI>
cardPlayers) {
        if (p1 instanceof DicePlayerI && p2 instanceof DicePlayerI) {
            diceGame((DicePlayerI) p1, (DicePlayerI) p2);
        }

        if (p1 instanceof CardPlayerI) {
            cardPlayers.add((CardPlayerI) p1);
        }

        if (p2 instanceof CardPlayerI) {
            cardPlayers.add((CardPlayerI) p2);
        }
    }

    static int cntAces(ArrayList<Card> cards) {
        int rVal = 0;
        for (Card card : cards) {
            if (card.getCardRank() == CardRank.ACE) {
                rVal++;
            }
        }
        return rVal;
    }

    public static void main(String[] args) {
        Person[] persons = new Person[14];
        persons[0] = new DicePlayer("100", "Jack", "Antie",
LocalDate.of(2001, Month.MARCH, 1));
        persons[1] = new DicePlayer("101", "John", "Travolta",
LocalDate.of(1961, Month.AUGUST, 3));
        persons[2] = new DicePlayer("102", "Bill", "Allen",
LocalDate.of(1980, Month.FEBRUARY, 4));
        persons[3] = new DicePlayer("103", "Jim", "Morison",
LocalDate.of(2001, Month.JULY, 5));
        persons[4] = new DicePlayer("104", "Peter", "MacMillan",
LocalDate.of(2002, Month.MARCH, 1));
        persons[5] = new DicePlayer("105", "George", "Best",
LocalDate.of(1970, Month.MAY, 1));
        persons[6] = new DicePlayer("106", "George", "Blue",
LocalDate.of(1980, Month.MAY, 1));
        persons[7] = new DicePlayer("107", "Beny", "Hill",
LocalDate.of(1977, Month.JULY, 21));
        persons[8] = new CardPlayer("108", "John", "Beker",
LocalDate.of(1977, Month.JULY, 23));
        persons[9] = new CardPlayer("109", "Alison", "Stair",
LocalDate.of(1978, Month.APRIL, 22));
        persons[10] = new CardPlayer("110", "Erik", "Erikson",
LocalDate.of(1979, Month.JANUARY, 13));
```

```
        persons[11] = new Gambler("111", "Erik", "Johnson",
LocalDate.of(1989, Month.JANUARY, 13));
        persons[12] = new Gambler("112", "Klint", "Eastwood",
LocalDate.of(1983, Month.FEBRUARY, 12));
        persons[13] = new Gambler("113", "Jane", "Fonda",
LocalDate.of(1989, Month.JANUARY, 13));

        Set<CardPlayerI> cardPlayers = new HashSet<>();

        for (int i = 0; i < persons.length - 1; i++) {
            for (int j = i + 1; j < persons.length; j++) {
                meet(persons[i], persons[j], cardPlayers);
            }
        }

        cardGame(cardPlayers);

        List<Entry<DicePlayerI, Integer>> sortedScores = new
LinkedList<>(diceScores.entrySet());
        Collections.sort(sortedScores, new Comparator<Entry<DicePlayerI,
Integer>>() {
            @Override
            public int compare(Entry<DicePlayerI, Integer> o1,
Entry<DicePlayerI, Integer> o2) {
                return o2.getValue().compareTo(o1.getValue());
            }
        }
        );
       for (Entry<DicePlayerI, Integer> entry:sortedScores) {
           System.out.println(entry.getKey()+"->"+entry.getValue());
       }
       System.out.println("");

        Map<CardPlayerI, ArrayList<Card>> opened = new LinkedHashMap<>();
        int maxAces = 0;
        for (CardPlayerI p : cardPlayers) {
            ArrayList<Card> cardsInHand = p.openCards();
            System.out.print(p + ":\n ");
            System.out.println(cardsInHand);
            opened.put(p, cardsInHand);
            int cAces = cntAces(cardsInHand);
            if (cAces > maxAces) {
                maxAces = cAces;
            }
        }

        ArrayList<CardPlayerI> winners = new ArrayList<>();
        for (CardPlayerI p : opened.keySet()) {
            int cAces = cntAces(opened.get(p));
            if (cAces == maxAces) {
                winners.add(p);
            }
        }
        System.out.println("---------------------------------------");
        System.out.println("winner in card Game: \n" + winners + " Aces: "
+ maxAces);
    }
}
```

Κώδικας 14.18 *Η κλάση BeckersMeeting*

Ας ξεκινήσουμε από την main της κλάσης. Καταρχάς, δημιουργούμε και φορτώνουμε στον πίνακα persons τα 14 πρόσωπα. Τα 8 είναι DicePlayer, τα 3 CardPlayer και τα υπόλοιπα 3 Gambler. Στη συνέχεια, ορίζουμε το cardPlayers τύπου HashSet. Το cardPlayers είναι ο κατάλογος στον οποίο εγγράφονται οι χαρτοπαίκτες σύμφωνα με την εκφώνηση. Η επιλογή του HashSet βασίστηκε στο γεγονός ότι κάθε χαρτοπαίκτης έχει νόημα να γραφτεί μόνο μία φορά. Αμέσως μετά υλοποιείται η απαίτηση να συναντηθούν τα πρόσωπα ανά δύο με τη βοήθεια της meet. Προσέξτε την εμφωλευμένη for μέσα στην οποία καλείται η meet. Οι συνδυασμοί ν αντικειμένων ανά 2 είναι ν×((ν-1))/2 , αυτό ακριβώς αποδίδει η for. Με άλλα λόγια, για κάθε ζεύγος προσώπων p1, p2 συναντάται το ζεύγος (p1, p2) μόνον εφόσον p1 διάφορο του p2, ενώ δεν επαναλαμβάνεται συνάντηση για το ζεύγος (p2, p1).

Αν μεταφερθούμε προσωρινά στη συνάρτηση meet, βλέπουμε πως καταρχάς ελέγχεται αν είναι και τα δύο πρόσωπα παίκτες ζαριού. Στην περίπτωση αυτή παίζουν μια παρτίδα ζάρια με την diceGame. Στη συνέχεια εφόσον είναι κάποιο από τα πρόσωπα χαρτοπαίκτης γίνεται η εγγραφή του στην ομάδα χαρτοπαικτών. Με το τέλος των συναντήσεων, διεξάγεται η παρτίδα χαρτιών με την cardGame.

Η diceGame ενημερώνει με το σκορ όλων των παιχνιδιών κάθε παίκτη την diceScores που είναι στατική μεταβλητή της κλάσης BeckersMeeting. Με την cardGame, ο κάθε χαρτοπαίκτης διατηρεί μια σειρά από κάρτες με βάση τις οποίες βγαίνουν οι νικητές.

Στη συνέχεια της main έχουμε να εμφανίσουμε τα αποτελέσματα. Θα πρέπει καταρχάς να ταξινομήσουμε τα αποτελέσματα των παιχνιδιών ζαριού ως προς φθίνουσα σειρά των σκορ. Εδώ μας βοηθάει η λίστα sortedScores. Κάθε στοιχείο αυτής της λίστας αναπαριστά ένα στοιχείο της diceScores. Στη συνέχεια ταξινομείται η λίστα με τη βοήθεια του Comparator που περνάμε ως παράμετρο στην Collections.sort. Προσέξτε πως ο Comparator δημιουργείται ως αντικείμενο ανώνυμης κλάσης. Επίσης, σημειώστε πως η compare του Comparator ταξινομεί κατά φθίνουσα σειρά. Έχοντας ταξινομημένα τα scores είναι εύκολο να τα εμφανίσουμε κατά φθίνουσα σειρά σύμφωνα με τις απαιτήσεις της εκφώνησης.

Τώρα θα πρέπει να ανοίξουν οι χαρτοπαίκτες τα χαρτιά που έχει ο κάθε ένας στο χέρι του, να γίνει καταμέτρηση και να ανακηρυχτεί ο νικητής ή οι νικητές. Οπότε κάθε ένας εγγεγραμμένος χαρτοπαίκτης ανοίγει τις κάρτες του που προσωρινά αποθηκεύονται στην τοπική μεταβλητή cardsInHand. Στη συνέχεια εμφανίζονται τα στοιχεία του παίκτη και τα χαρτιά του. Επιπλέον, τα χαρτιά κάθε παίκτη εισάγονται στην opened καθώς χρειάζονται για την τελική καταμέτρηση και ανάδειξη των νικητών. Παράλληλα με όλη αυτή τη διαδικασία υπολογίζεται ο μέγιστος αριθμός άσων που υπάρχουν στα χαρτιά κάθε χαρτοπαίκτη. Όλοι όσοι έχουν τον ίδιο αριθμό άσων θα πρέπει να ανακηρυχτούν νικητές.

Τέλος, εντοπίζονται οι νικητές της χαρτοπαιξίας και εμφανίζονται τα στοιχεία τους.

14.5.3 Πίνακας χημικών στοιχείων

Ο περιοδικός πίνακας των χημικών στοιχείων [3] περιλαμβάνει όλα τα γνωστά χημικά στοιχεία κατηγοριοποιημένα σε ομάδες, περιόδους και τομείς. Πιο συγκεκριμένα, υπάρχουν 18 ομάδες, 7 περίοδοι και 4 τομείς. Οι ομάδες αριθμούνται από 1 έως 18, οι περίοδοι από 1 έως 7, ενώ οι τομείς χαρακτηρίζονται με τους χαρακτήρες s, p, d, f. Κάθε χημικό στοιχείο ανήκει σε έναν τομέα, σε μία ομάδα και σε μία περίοδο. Επιπλέον, κάθε χημικό στοιχείο έχει ένα σύμβολο αναφοράς, π.χ. Η για το υδρογόνο, Na για το Νάτριο, καθώς και έναν ατομικό αριθμό.

Να αναπτυχθεί η κλάση ChemicalElements κάθε αντικείμενο της οποίας αναπαριστά ένα χημικό στοιχείο. Επιπλέον, να αναπτυχθεί η κλάση TableOfChemichalElements που αναπαριστά πίνακες χημικών στοιχείων και διαθέτει τις μεθόδους:

```
public boolean add(ChemicalElement cE)
```

Αν το στοιχείο cE, υπάρχει στον πίνακα επιστρέφει false. Αν δεν υπάρχει, το εισάγει και επιστρέφει true.

```
public TableOfChemichalElements getGroup(int groupId)
```

Επιστρέφει έναν πίνακα που περιλαμβάνει τα στοιχεία αυτού του πίνακα που ανήκουν στην ομάδα groupId.

```
public TableOfChemichalElements getPeriod(int periodId)
```

Επιστρέφει έναν πίνακα που περιλαμβάνει τα στοιχεία αυτού του πίνακα που ανήκουν στην περίοδο periodId.

```
public TableOfChemichalElements getSection(char sectionId)
```

Επιστρέφει έναν πίνακα που περιλαμβάνει τα στοιχεία αυτού του πίνακα που ανήκουν στον τομέα sectionId.

```
public boolean isEmpty()
```

Ελέγχει αν αυτός ο πίνακας είναι κενός.

```
public Iterator groupIterator(int groupId)
```

Επιστρέφει έναν Iterator στα στοιχεία της ομάδας με groupId. Ο Iterator υποστηρίζει εκτός από την hasNext και next και την remove. Η remove διαγράφει το τελευταίο στοιχείο που επέστρεψε η next. Δείτε τη διεπαφή της remove στην τεκμηρίωση του Iterator.
Προσθέστε main με βασικό τεστ κώδικα της κλάσης TableOfChemichalElements.

Λύση
Στον κώδικα 14.19, παρατίθεται η κλάση ChemicalElement.

```
public class ChemicalElement {

    private final String symbol;
    private final int atomicNumber;
    private final int group;
    private final int period;
    private final char section;

    public ChemicalElement(int group, int period, char sector, int
atomicNumber, String symbol) {
        this.group = group;
        this.period = period;
        this.section = sector;
        this.atomicNumber = atomicNumber;
        this.symbol = symbol;
    }

    public int getGroup() {
        return group;
    }

    public int getPeriod() {
        return period;
    }

    public char getSection() {
        return section;
    }

    public int getAtomicNumber() {
        return atomicNumber;
    }

    public String getSymbol() {
        return symbol;
    }

    @Override
    public int hashCode() {
        int hash = 7;
        hash = 89 * hash + this.symbol.hashCode();
        return hash;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
```

```
            return true;
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
        }
        final ChemicalElement other = (ChemicalElement) obj;
        return this.symbol.equals(other.symbol);
    }

    @Override
    public String toString() {
        return symbol + ", AN=" + atomicNumber + ", group=" + group + ",
period=" + period + ", sector=" + section;
    }
}
```

Κώδικας 14.19 *Η κλάση ChemicalElement*

Η κλάση ChemicalElement είναι τυπική κλάση και περικλείει όλες τις αναγκαίες μεθόδους. Στον κώδικα 14.20 παρατίθεται η κλάση TableOfChemichalElements.

```
import java.util.ArrayList;
import java.util.Iterator;

public class TableOfChemichalElements {

    private final ArrayList<ChemicalElement> store = new ArrayList<>();

    public class GroupIterator implements Iterator {

        private final int groupId;
        private int current = 0;

        public GroupIterator(int groupId) {
            this.groupId = groupId;
        }

        @Override
        public boolean hasNext() {

            while (current < store.size()) {
                ChemicalElement cE = store.get(current);
                if (cE.getGroup() == groupId) {
                    return true;
                }
                current++;
            }
            return false;
        }

        @Override
        public Object next() {
            return store.get(current++);
        }

        @Override
        public void remove() {
            store.remove(--current);
```

```
        }
    }

    public Iterator groupIterator(int groupId) {
        return new GroupIterator(groupId);
    }

    public boolean add(ChemicalElement cE) {
        if (store.contains(cE)) {
            return false;
        }
        return store.add(cE);
    }

    public TableOfChemichalElements getGroup(int groupId) {
        TableOfChemichalElements rVal = new TableOfChemichalElements();
        for (ChemicalElement cE : store) {
            if (cE.getGroup() == groupId) {
                rVal.add(cE);
            }
        }
        return rVal;
    }

    public TableOfChemichalElements getPeriod(int periodId) {
        TableOfChemichalElements rVal = new TableOfChemichalElements();
        for (ChemicalElement cE : store) {
            if (cE.getPeriod() == periodId) {
                rVal.add(cE);
            }
        }
        return rVal;
    }

    public TableOfChemichalElements getSection(char sectionId) {
        TableOfChemichalElements rVal = new TableOfChemichalElements();
        for (ChemicalElement cE : store) {
            if (cE.getSection() == sectionId) {
                rVal.add(cE);
            }
        }
        return rVal;
    }

    public boolean isEmpty() {
        return store.isEmpty();
    }

    @Override
    public String toString() {
        StringBuilder t = new StringBuilder();
        for (ChemicalElement cE : store) {
            t.append(cE).append("\n");
        }
        return t.toString();
    }

    public static TableOfChemichalElements setUp() {
        ChemicalElement h = new ChemicalElement(1, 1, 's', 1, "H");
        ChemicalElement li = new ChemicalElement(1, 2, 's', 3, "Li");
        ChemicalElement na = new ChemicalElement(1, 3, 's', 11, "Na");
```